

Eliciting Requirements for a Conversational Agent to Assist the Software Staging Process in DevOps

1st David Mosquera

Zurich University of Applied Sciences
Winterthur, Switzerland
mosq@zhaw.ch

2nd Marcela Ruiz

Zurich University of Applied Sciences
Winterthur, Switzerland
marcela.ruiz@zhaw.ch

3rd Leandra Willie

Zurich University of Applied Sciences
Winterthur, Switzerland
leandra.willi@zhaw.ch

4th Thomas Riedel

Swiss Digital Network
Zurich, Switzerland
thomas.riedel@digitalq.ch

5th Olivier Mann

Swiss Digital Network
Zurich, Switzerland
olivier.mann@swiss-digital-network.ch

6th Makram Hanin

Swiss Digital Network
Zurich, Switzerland
makram.hanin@swiss-digital-network.ch

Abstract—Context. Staging environments are implemented by DevOps teams who follow a staging process encompassing two main tasks: 1) **Data collection:** Determining the necessary data from software requirements, test results, and observability; in order to perform 2) **Evaluation:** Judge whether functional and non-functional requirements are met for production. **Problem.** In practice, the main tasks of software staging processes are heavily based on human experience and manual interventions that introduce delays and inconsistencies. Despite the promise of large language model (LLM)-based conversational agents for supporting DevOps teams with data collection and evaluation, there is a lack of industry-proven requirements to guide the effective design and integration of conversational agents in software staging. **Method.** To solve this problem, we elicited 135 requirements from eight senior software staging experts through focus group sessions. **Results.** After applying topic-based clustering and human-in-the-loop refinement, we elicited questions, expected behaviours, and usefulness contexts for a conversational agent, ensuring explainable, evidence-based, and human-centred features. **Conclusion.** We provide a set of actionable industry-elicited requirements and a multi-agent architecture to guide the implementation of a conversational agent for decision-making in software staging. We envision our results as a baseline to implement proofs-of-concept adapted to different industry domains.

Index Terms—Requirements Engineering for Conversational Agents; Software Staging Process; DevOps; Multi-agent architecture; Site Reliability Engineering

I. INTRODUCTION

Modern software delivery pipelines increasingly rely on continuous integration and continuous deployment (CI/CD) to shorten release cycles and accelerate value delivery. Staging environments are created by DevOps teams as high-fidelity replicas of production conditions to determine whether the new software versions are still reliable in production environments. For this, *decision-making in the staging process*—the go/no-go evaluations that determine whether software can

safely progress across staging environments—have become both more frequent and more relevant.

The staging process comprises two main tasks: 1) determining and analysing the necessary data from software requirements, test results, performance telemetry, observability traces, historical executions, and service-level compliance data; and 2) judging the software quality and reliability based on such data—i.e., evaluating whether functional and non-functional requirements are met for production [1]. The second task, which is inherently dependent on the first, is what we refer to as decision-making within the software staging process.

Although DevOps is highly automated nowadays, software quality information remains fragmented across tools, teams, and lifecycle stages [1]. DevOps teams must manually correlate distributed information under strict time pressure, often relying on tacit knowledge rather than consistent, evidence-based reasoning to complete the staging process.

This fragmentation introduces significant challenges for DevOps teams’ roles like quality assurance (QA) engineers, site reliability engineers (SREs), test leads, and release managers [1]. Prior research shows that, even when relevant data is available, the cognitive effort required to synthesise it into a coherent staging recommendation during the decision-making phase remains high, leading to conservative decisions, protracted handoffs, or insufficiently justified releases [2], [3].

At the same time, advances in large language models (LLMs) and conversational agents have created new opportunities for assisting decision-making. Recent studies suggest that conversational agents can lower cognitive load, facilitate cross-tool navigation, increase transparency, and offer role-specific explanations for complex operational data [4], [5].

The **AI-SQUARE** [1] industry innovation project funded by the Swiss Innovation Agency Innosuisse, is dedicated to advancing decision-assisted technologies in software staging management. AI-SQUARE aims to ease the staging process by leveraging a knowledge-graph-centric platform that unifies requirements, test artefacts, observability data, anomalies, and risk signals, among other data [6]. The AI-SQUARE project is

This research has been funded by the Swiss Innovation Agency Innosuisse in collaboration with the Swiss Digital Network in the context of the AI-SQUARE project: www.ai-square.ch.

closely linked to industry, especially in Switzerland’s financial and banking sector, where high software quality and reliability are a must. As part of the AI-SQUARE main goals, we explore how to design and implement a conversational agent to assist decision-making in the software staging process. We focus on conversational agent technology due to its disruptive and potential impact in the software engineering field, including software staging [7].

The main industry-relevant research challenge we found is: *What are the functional and non-functional requirements expected by industry for conversational agents assisting decision-making in software staging processes?* To address this, we propose two research questions:

- **(RQ1)** What questions and behaviours should a conversational agent exhibit for decision-making in software staging processes? and
- **(RQ2)** How to design a conversational agent that satisfies usefulness requirements in an industry context?

To answer the research questions, we conducted focus groups with eight senior software staging experts from the financial and banking sector who are part of DevOps teams. As a result of the focus group, we elicited 135 functional and non-functional requirements for a conversational agent. The empirical results were analysed using topic-based clustering. In general, the results show that DevOps teams expect conversational agents to assist them with: i) **understanding decisions** in the staging process; ii) **achieving confidence** when making decisions in the staging process; and iii) **ensuring decision consistency** based on previous decisions.

Based on these findings, this paper makes three contributions: i) We report industry-driven functional and non-functional requirements for a conversational agent to assist with decision-making in the software staging process, including requirements regarding questions that must be answered and expected behaviour; ii) We provide industry requirements regarding the usefulness context for a conversational agent, together with an iStar 2.0 goal model [8] summarising the dependencies among conversational agents and DevOps teams. For this, we also provide design features of the conversational agent regarding how it should behave, what motivates it, and which tasks it must assist; and iii) We propose a reference LLM-based multi-agent architecture that instantiates the elicited requirements in the context of AI-SQUARE.

The remaining part of this paper is structured as follows: in Section II we review related work and motivate our research questions; in Section III we introduce our focus group protocol; in Section IV we present functional and non-functional requirements to answer RQ1; in Section V we present a goal-model for software staging assistance and a multi-agent reference architecture to answer RQ2; in Section VI we present threats to validity, and, finally, in Section VII we draw conclusions and next steps.

II. BACKGROUND AND MOTIVATION

A. Background

Related work has reviewed the challenges in modern DevOps and continuous delivery pipelines. Korkmaz et al. [9] concluded that integrating DevOps tools with data across teams can improve collaboration, increase deployment frequency, and reduce incident rates, highlighting the importance of coordinated pipelines for reliable releases. Similarly, Olsson et al. [10] show that inconsistent or inefficient handling of quality requirements often results in delayed staging decisions and insufficient quality coverage—directly affecting release readiness. These findings indicate that tool-supported approaches are essential for effective software staging processes.

At the same time, other studies [11] report that performance and quality-evaluation tools remain complex, fragmented, and poorly integrated into DevOps workflows, limiting their value for decision-making within the software staging process. Amaro et al. [12] emphasise that meaningful adoption of such tools requires end-to-end support across data sources, along with improved collaboration and communication among stakeholders engaged in evidence-based decision-making.

Advances in AI and AIOps have introduced new opportunities for tool support. Some authors [13] have reported improvements in anomaly detection, incident reduction, and predictive fault management through machine learning techniques in the staging process. However, there are still challenges to be addressed [14]: the lack of architectures capable of unifying heterogeneous data, supporting reasoning, and producing explainable recommendations in the staging process.

These gaps motivate the AI-SQUARE innovation project and platform [1], [6], which aims to unify heterogeneous lifecycle data and enable explainable, context-aware analysis to assist the staging process more effectively.

B. The AI-SQUARE Innovation project and Platform

The AI-SQUARE innovation project and platform [6] is an intelligent, end-to-end software staging assistance system designed to help DevOps teams manage their staging processes. The software staging process—i.e., determining whether software may safely progress from staging environments such as development to testing or production—is executed under time pressure and based on large, heterogeneous, and interconnected software quality data. This data ranges from requirements and test cases to performance metrics, logs, traces, and previous release outcomes.

As described in Section II-A, the staging process requires DevOps teams to interpret ubiquitous and deeply interrelated data manually, relying heavily on experience to compensate for the lack of integrated, cross-source reasoning. To address this, AI-SQUARE introduces a knowledge-graph-based architecture that unifies diverse data sources across the entire software lifecycle—including requirements, tests, observables, anomalies, and quality-gate rules—thereby providing a shared “lingua franca” to support end-to-end reasoning.

Similarly to AI-SQUARE, some other authors have used taxonomies [15] and ontologies to produce toolkits for events

and anomaly analysis. Others have used knowledge graphs [16]–[18] for root cause analysis and anomaly detection in cloud applications, focusing mainly on microservice applications. Other commercial tools have assisted reliability engineers by providing data-specific analysis, such as correlations between performance metrics [19]. Nevertheless, the AI-SQUARE platform provides end-to-end assistance for the software staging process by i) combining multiple data sources and ii) enabling judgment of software quality and reliability based on them—i.e., assisting decision-making.

Architecturally, the AI-SQUARE platform comprises five layers: the data layer, the AI layer, the staging decision process, the access control layer, and the user interaction layer. However, assisting decision-making in the software staging process with such interconnected data from requirements to software quality outcomes requires an interface that lowers cognitive load while maintaining traceability and explainability. LLM-based conversational agents have demonstrated the ability to navigate and query large-scale, heterogeneous data, ask natural-language questions, and obtain cross-source explanations in software engineering (see Section II-C).

C. Related work: LLM-based conversational agents for the staging process

Several authors have proposed LLM-based conversational agents to assist with software engineering tasks, including code generation, requirements engineering, design, deployment, and more [20]. Cai et al. [21] provide a literature review quantifying this trend, showing that code generation dominates, representing 47.9% of all proposals, while software staging accounts for only 1.1%. These results highlight that the application of conversational agents to software staging and deployment **remains unexplored**.

Among the approaches for software staging, some authors have proposed GoNoGo [22] and GateLens [23]. These approaches introduce LLM-based architectures designed to assist the staging process in automotive software deployment, satisfying domain-specific functional requirements and safety constraints. However, these approaches primarily operate on structured tabular data and are tailored to the automotive domain. As a result, they do not address the broader complexity of heterogeneous data sources relevant to other domains—such as logs, test results, performance metrics, and cross-system quality attributes—present in the AI-SQUARE platform.

D. Motivation and Research Questions

In view of the gaps in the literature and the challenges observed in the context of the AI-SQUARE platform, we identify the following main research question (MRQ):

MRQ: What are the functional and non-functional requirements expected by industry for a conversational agent assisting decision-making in software staging processes?

To address our MRQ, we propose two research questions:

- **RQ1: What questions and behaviours should a conversational agent exhibit for decision-making in software staging processes?** With RQ1, we aim to extract the point of view of industry experts on the software staging landscape, acquiring functional and non-functional requirements for a conversational agent. We show the results in Section IV.
- **RQ2: How to design a conversational agent that satisfies usefulness requirements in an industry context?** With RQ2, we aim to provide a design that meet the usefulness requirements from industry and reflects the results from RQ1. Thus, we propose a goal and a reference architecture to guide the implementation of conversational agents for supporting staging decisions. We provide such designs in Section V.

III. FOCUS GROUP DESIGN

Following the Goal/Question/Metric template [24], the goal of our experiment is to **analyse the requirements of a conversational agent for the purpose of assisting software staging decisions with respect to questions to be answered, expected behaviours, and perceived usefulness from the point of view of DevOps team roles: SREs, performance engineers, and QA industry experts in the context of the AI-SQUARE innovation project**.

To address this goal, we use focus groups to elicit tacit knowledge, shared heuristics, and cross-role priorities, following qualitative research guidelines in software engineering [25], [26]. Although the AI-SQUARE platform is designed to be domain-independent, its current adoption is concentrated in the financial and banking sectors in Switzerland. Therefore, the participating DevOps roles reflect this domain context, which frames the scope of our findings.

A. Subjects

We recruited eight industry practitioners responsible for decision-making in the software staging process: an Expert in Continuous Verification; Head of Test Engineering; Site Reliability Engineer; Non-Functional Quality Engineering (NFQE) capability lead; Test Engineering Lead; Observability Architect and SRE Consultant; DevOps and SRE Consultant; and Software Architect. Subjects were recruited through purposive sampling, i.e., they were intentionally selected within the industrial context of the AI-SQUARE project because they were relevant stakeholders as potential users of the envisioned conversational agent. We contacted them via direct email messages. Of the 12 contacted subjects, 8 expressed interest in participating in the focus group. No financial or material incentive was offered. Participants' motivation was primarily the opportunity to contribute to the design of solutions that they may use in the future to support their daily work. To characterise our subjects, we used an experience-based segmentation. On average, subjects reported 17.3 ± 10.3 years of industry experience; seven of eight (87.5%) self-identified as senior (≥ 5 years), and one (12.5%) as experienced (2–5

years). Four participants worked in large organisations ($\geq 1,000$ employees) and four in medium-sized organisations (10–50 employees). Participants were proficient with software quality assurance tools used for software staging, including Micro Focus ALM/Quality Center, ServiceNow, SonarQube, GitLab, Grafana, Dynatrace, Selenium, LoadRunner, Apache JMeter, Jira, and Squash (among others). Four participants also had experience as full-stack developers in addition to their QA roles. Only one subject reported prior experience building machine learning and AI systems. On the other hand, seven of eight subjects reported using LLM-based conversational agents at work—primarily Microsoft Copilot and ChatGPT—though none reported using LLMs to assist decision-making in staging processes.

B. Variables

We consider three qualitative variables (QV):

- **(QV1) Questions a conversational agent must answer to assist decision-making in software staging processes.** We capture text quotes from industry experts describing the functional requirements for such an agent.
- **(QV2) How the conversational agent is expected to behave.** We capture text quotes describing non-functional requirements for decision-making assistance in staging processes (e.g., reliability, transparency, traceability, latency, and privacy).
- **(QV3) Situations in which the conversational agent is considered useful or not useful.** We capture text quotes that describe the situations and scenarios where a conversational agent is perceived as useful or not in decision-making.

C. Session Execution

We conducted three focus group sessions, each 90 ± 15 -minute session followed an experimental protocol: i) informed consent; ii) short motivation, a demo, and orientation about software staging and the AI-SQUARE platform; and iii) a structured discussion across three questions: (1) *Questions* the assistant must/should/could/will-not answer (functional requirements); (2) *Expected qualities and behaviours* from the conversational agent (non-functional requirements); and (3) *Situations* where assistance is useful or not. We show the session execution graphically in Figure 1.

Participants were split into sessions based on availability. The orientation introduced the notion of staging adopted in this study and was used to establish a shared vocabulary between researchers and subjects. The subsequent discussion of current practices helped identify how subjects understood and implemented staging in their organisational contexts before eliciting requirements for the assistant.

We collected insights on the variables (QV1, QV2, and QV3) by having participants write individual statements on post-its first, to reduce early group influence. The statements were then discussed collectively and placed in the corresponding MoSCoW quadrant (*Must*, *Should*, *Could*, or *Will not*) [27], [28]. The moderator asked participants to clarify unclear

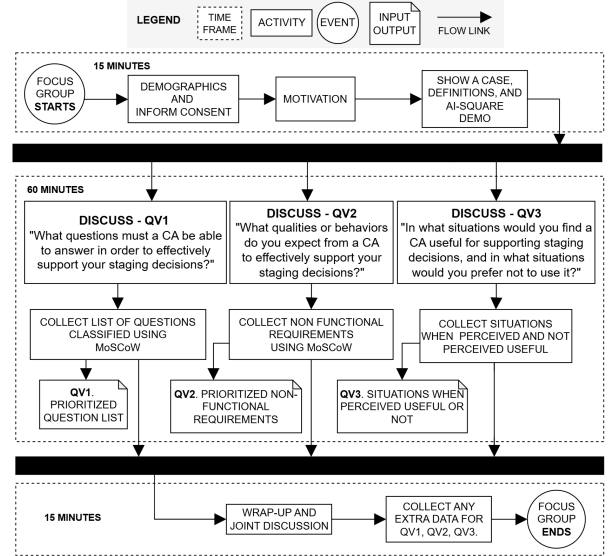


Fig. 1. Focus Group Session Execution.

statements, encouraged contributions from all participants, and explicitly checked for agreement before retaining the final placement.

The demos during the session were used solely as running examples to make the envisioned interaction concrete and to initiate discussion. To reduce the risk of biasing participants toward the demonstrated functionality, moderators emphasised that the demo was illustrative rather than prescriptive, and explicitly invited participants to propose needs, concerns, and situations beyond what was shown. We collected the post-its and complemented them with session transcripts to preserve both the resulting statements and the rationale discussed during the sessions.

D. Results and Data Analysis

Following our focus group protocol, we extracted a total of 135 individual requirements from QV1, QV2, and QV3. These individual requirements were prioritized using the MoSCoW requirements prioritization method [27], [28] (QV1 and QV2) or useful/not useful (QV3) depending on the QV. Then, we performed topic-based clustering using BERTopic, a neural topic modelling tool based on a class-based TF-IDF procedure [29], a tool for short-text and embedding-driven topic analysis. The resulting topic-based clusters were then refined through a human-in-the-loop process, with the first author reviewing and adjusting them. Subsequently, the second and third authors reviewed the cluster definitions and representative quotes. To preserve the original priority assigned to each individual requirement by the participants, we computed the priority of each cluster as the average rating of the contributions it contained, using a numerical encoding of the MoSCoW categories from 1 = ‘Will not’ to 4 = ‘Must’. We report the threats to validity of our results and data analysis in Section VI. To support transparency and reproducibility, we provide

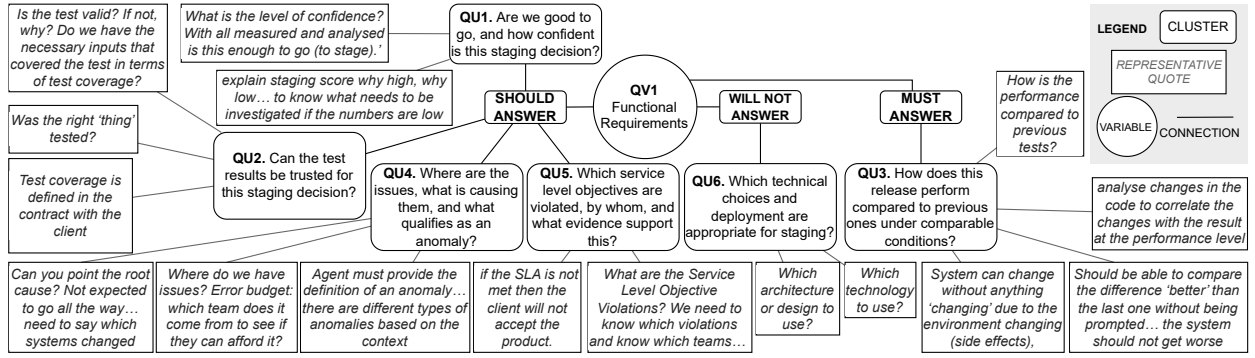


Fig. 2. QV1 Questions a conversational agent must/should/will not answer: Clusters and textual quotes.

a public repository containing the focus group protocol, the analysis scripts, and anonymised raw data [30].

IV. INDUSTRY-DRIVEN REQUIREMENTS FOR A CONVERSATIONAL AGENT TO ASSIST SOFTWARE STAGING PROCESSES

To answer RQ1, we present and analyse the clusters related to QV1 and QV2—i.e., functional and non-functional requirements expected by the industry experts that participated in our focus groups.

A. (QV1) Questions to be answered

To collect data for QV1, we analysed 61 individual functional requirements from subjects across focus groups. This analysis resulted in six clusters about questions (QU) to be answered by a conversational agent: **QU1** Are we good to go, and how confident is this staging decision?; **QU2** Can the test results be trusted for this staging decision?; **QU3** How does this release perform compared to previous ones under comparable conditions?; **QU4** Where are the issues, what is causing them, and what qualifies as an anomaly?; **QU5** Which service level objectives are violated, by whom and what evidence supports this?; **QU6** Which technical choices and deployment are appropriate for staging? These questions are classified as must, should, could, and will not be answered (see Figure 2).

1) Must Answer

QU3. How does this release perform compared to previous ones under comparable conditions? This cluster focuses on subjects' questions comparing one or more releases, avoiding isolated performance analysis. It addresses the comparability of results when staging environments differ, the provision of a validity score for comparisons, and the use of historical data to establish meaningful baselines. As stated in the requirements, subjects expect the conversational agent to answer: "How is the performance compared to previous tests? Validity of the test before the [staging] environment might not be the same. It is important to provide a validity score [or assessment]." Subjects highlight the need to compare results within the same staging environment—"The conversational agent] should be able to compare the difference [to the previous staging environment]

... without being prompted"—while acknowledging that different environments introduce side effects that may obscure performance changes. QU3 is positioned as a "must-answer" cluster according to the MoSCoW-based classification. On a numeric scale from 1 ("Will not answer") to 4 ("Must answer"), QU3 obtains an average priority score of 3.5 ± 0.8 .

2) Should answer

QU1. Are we good to go, and how confident is this staging decision? This cluster addresses subjects' questions related to determining whether a software is sufficiently prepared to progress to the next staging environment and assessing the confidence associated with the decision-making. The cluster focuses on explaining confidence levels, data sufficiency, metric applicability, and the reasoning behind go/no-go recommendations. As expressed in the requirements, subjects ask: "What is the level of confidence? With all [data] measured and analysed is this enough to go [to stage]... basically, how certain is the [conversational agent] tool that they [i.e., the software under evaluation] should go, giving a certain score and [give reasoning] why when 'not go.'" This includes identifying which data sources and categories influence the decision, providing high-level breakdowns of functional, performance, and other quality dimensions, and clarifying when certain data are missing or not applicable. QU1 is positioned as a "should answer" priority cluster according to the MoSCoW-based classification. On a numeric scale from 1 ("Will not answer") to 4 ("Must answer"), QU1 obtains an average priority score of 3.2 ± 0.8 .

QU2. Can the test results be trusted for this staging decision? This cluster concerns subjects' questions about the trustworthiness of test results as evidence for making a decision, focusing on the validity, completeness, and appropriateness of the tests and the reliability of the underlying data. It addresses whether the tests are valid, whether the necessary inputs and requirements have been sufficiently covered, and whether the staging environment used to obtain the measurements was stable enough to yield trustworthy results. As reflected in the requirements, stakeholders explicitly ask: "Is the test valid? If not, why? Do we have the necessary inputs that covered the test in terms of test coverage?" Overall, this cluster encapsulates the multi-dimensional reasoning required

to judge whether test outcomes constitute credible, sufficient evidence for making a decision in a software staging process. QU2 is positioned as a "should answer" cluster according to the MoSCoW-based classification. On a numeric scale from 1 ("Will not answer") to 4 ("Must answer"), QU2 obtains an average priority score of 3.4 ± 0.8 .

QU4. Where are the issues, what is causing them, and what qualifies as an anomaly? This cluster captures subjects' questions related to identifying issues that affect software readiness, understanding their root cause, and distinguishing true anomalies from expected variation. It includes determining where these issues occur and how they trace back to error budgets—i.e., the permissible level of reliability degradation relative to defined service objectives—reflecting the requirement: "Where do we have issues? Error budget: which team does it come from to see if they can afford it?" The cluster focuses on staging-environment-level root cause analysis that does not require pinpointing specific lines of code but should still help teams narrow down problematic sources or components. QU4 is positioned as a "should-have" priority cluster according to the MoSCoW-based classification. On a numeric scale from 1 ("Will not answer") to 4 ("Must answer"), QU4 obtains an average priority score of 3.0 ± 1.0 .

QU5. Which service level objectives are violated, by whom, and what evidence supports this? This cluster addresses subjects' questions related to identifying which requirements—*a.k.a.* service level objectives—are violated, which teams are responsible for them, and what concrete evidence supports these determinations, forming a critical input to making a decision. It captures questions about detecting both active and resolved violations—consistent with the requirement: "What are the Service Level Objective Violations? We need to know which teams [are responsible for the SLO violations]... [as well to be able to] detect solved violations." It further encompasses explaining why these service level objectives are not satisfied. QU5 is positioned as a "should-answer" priority cluster according to the MoSCoW-based classification. On a numeric scale from 1 ("Will not answer") to 4 ("Must answer"), QU5 obtains an average priority score of 3.2 ± 0.8 .

3) Will not answer

QU6. Which technical choices and deployment are appropriate for staging? This cluster concerns subjects' questions about choosing appropriate technologies, architectures, designs, deployment strategies, and low-level remediation actions during the software staging process. Subjects do not expect the conversational agent to solve technical choices or design decisions, such as "Which technology to use?" and "Which architecture or design to use?" Consequently, QU6 is positioned as a "will not answer" cluster according to the MoSCoW-based classification. On a numeric scale from 1 ("Will not answer") to 4 ("Must answer"), QU6 obtains an average priority score of 1.4 ± 0.5 .

B. (QV2) Expected behaviours

To collect data for QV2, we analysed 42 individual non-functional requirements from subjects across focus groups.

This analysis resulted in six clusters about expected behaviours (BH) for the conversational agent: **BH1** Responsiveness, Efficiency, and Cost predictability; **BH2** Traceable quality evaluation and context-aware data handling; **BH3** Explainable, Evidence-based, and consistent decision support; **BH4** Secure data governance and controlled configuration; **BH5** Interaction Transparency and Multi-Modal Result presentation; **BH6** Enterprise interoperability and collaboration automation. These behaviours are classified as must, should, could, and will not have (see Figure 3).

1) Must have

BH3. Explainable, Evidence-based, and consistent decision support. This cluster specifies qualities required for trustworthy decision support, emphasising role-aware explanations, drill-down, and evidence-based answers. Based on subjects' requirements, the conversational agent should "explain the issue at hand... to business people as they don't understand the reasoning why something is an anomaly," adapting to stakeholder roles so explanations are appropriate. Conclusions must be justified—"not provide decisions without evidence." The agent should "avoid hallucinations [and] admit it has no answer," asking for clarification when questions are ambiguous. The subjects also require stability, as the conversational agent should "not change its decision during conversation... [not] be 'hacked' to change the [staging] decision after several iterations." BH3 is positioned as a "must-have" priority cluster according to the MoSCoW-based classification. On a numeric scale from 1 ("Will not answer") to 4 ("Must answer"), BH3 obtains an average priority score of 3.8 ± 0.4 .

2) Should have

BH1. Responsiveness, efficiency, and cost predictability. This cluster defines behaviours concerning response time, efficiency, and operational cost. It specifies acceptable response times that scale with query complexity, requiring "respond in less than 30 seconds for most queries... complex queries can be in minutes but not 10 min". The cluster constrains resource consumption and cost volatility, reflecting that the conversational agent "should not require huge amount of resources to be deployed", providing "transparency in resources and the cost." BH1 is positioned as a "should-have" priority cluster according to the MoSCoW-based classification. On a numeric scale from 1 ("Will not answer") to 4 ("Must answer"), BH1 obtains an average priority score of 3.3 ± 0.5 .

BH2. Traceable quality evaluation and context-aware data handling. This cluster captures the agent's ability to assess and communicate data, test, and requirement quality in a transparent, traceable, and context-aware way. It focuses on quantifying coverage and quality across requirements, tests, data, code, and sources, and on explicitly identifying gaps. Subjects mention that "if there is no [enough] data... that means there is a failure and a lack of quality on the test." The cluster also emphasises traceability from the agent, including "explain reasoning steps and sources... see the stages that the agent goes through" and the ability to suggest where more precise data is needed. BH2 is positioned as a "should-have" priority cluster according to the MoSCoW-based classification.

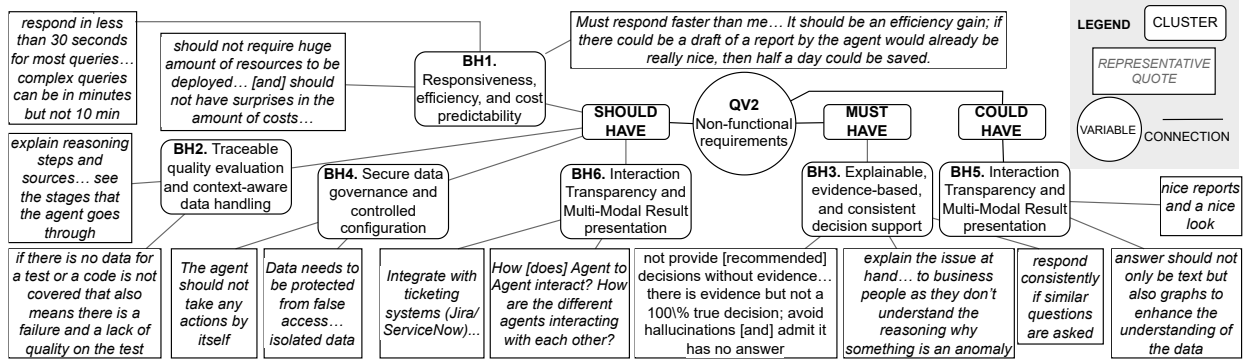


Fig. 3. QV2 Conversational agent non-functional requirements: Clusters and textual quotes.

On a numeric scale from 1 (“Will not answer”) to 4 (“Must answer”), BH2 obtains an average priority score of 3.4 ± 0.7 .

BH4. Secure data governance and controlled configuration. This cluster defines behaviours related to data protection, access governance, and system configuration. Subjects mentioned that the agent requires to enforce “least privilege” principles, ensuring that “data needs to be protected from false access... [data should be] isolated” and accessed only through properly defined permissions. The agent must be centralised and configured as subjects mentioned: “[the agent should be] centrally configured (e.g., temperature, hyperparameters)... so it cannot be changed per person”. Central configuration prevents per-user manipulation and improves auditability. BH4 is positioned as a “should-have” priority cluster according to the MoSCoW-based classification. On a numeric scale from 1 (“Will not answer”) to 4 (“Must answer”), BH4 obtains an average priority score of 3.3 ± 1.0 .

BH6. Enterprise interoperability and collaboration automation. This cluster defines behaviours related to enterprise-level integration and collaborative automation across systems and agents. It emphasises seamless interoperability with organisational platforms such as ticketing and notification systems. Subjects mention that the agent should “integrate with ticketing systems (Jira/ServiceNow)... [and be able to send] notifications if the evaluation fails.” Beyond system-to-system integration, the subjects mentioned that coordination among other agents is expected. Such agent-to-agent collaboration enables cross-application workflows and shared intelligence, expanding the value of the agent from isolated use to organisation-wide operational support. BH6 is positioned as a “should-have” priority cluster according to the MoSCoW-based classification. On a numeric scale from 1 (“Will not answer”) to 4 (“Must answer”), BH6 obtains an average priority score of 3.0 ± 1.0 .

3) Could have

BH5. Interaction Transparency and Multi-Modal Result presentation. This cluster captures qualities related to how the agent communicates its processing state and how it presents results to users, prioritizing transparency, clarity, and multi-modal expressiveness. The agent should “generate charts

to illustrate” results and combine graphical elements with concise textual explanations, as “answer should not only be text but also graphs to enhance the understanding of the data.” This includes data waiver controls—i.e. mechanisms that allow specific data points, anomalies, or violations to be explicitly excluded or temporarily accepted without impacting the overall staging decision. This is highlighted by subjects stating that “if there is an error, the agent should ask if we can ignore it or not... [to know if an error] should not play into the decision.” BH5 is positioned as a “could-have” priority cluster according to the MoSCoW-based classification. On a numeric scale from 1 (“Will not answer”) to 4 (“Must answer”), BH5 obtains an average priority score of 2.4 ± 0.8 .

V. CONVERSATIONAL AGENT DESIGN THAT SATISFIES USEFULNESS REQUIREMENTS IN AN INDUSTRIAL CONTEXT

To answer RQ2, we first present the clusters related to situations in which a conversational agent is considered useful or not useful in industry (i.e., QV3). Then, we propose a conversational agent design where we: 1) use the iStar (*i**) 2.0 language [8] to propose a goal model that reflects such situations and unify the industry-driven requirements elicited in RQ1; and 2) propose a reference LLM-based multi-agent architecture to show a technical implementation of our results in the context of the AI-SQUARE industry innovation project.

A. (QV3) Situations When a Conversational Agent is Perceived as Useful in Industry

We analysed 32 individual requirements from subjects across focus groups related to QV3. This analysis resulted in five clusters about situations (S) where a conversational agent is perceived useful or not for: **S1** When a Final Staging Decision Requires Human Accountability; **S2** When Non-Technical Stakeholders Need an End-to-End Explanation; **S3** When Assessing Test Readiness or Interpreting Test Outcomes; **S4** When Organizational or Technical Preconditions Limit Agent Value; **S5** When Large-Scale Data Comparison or Dependency Reasoning Is Needed. These situations are classified as useful or not useful (see Figure 4).

1) Perceived as Useful

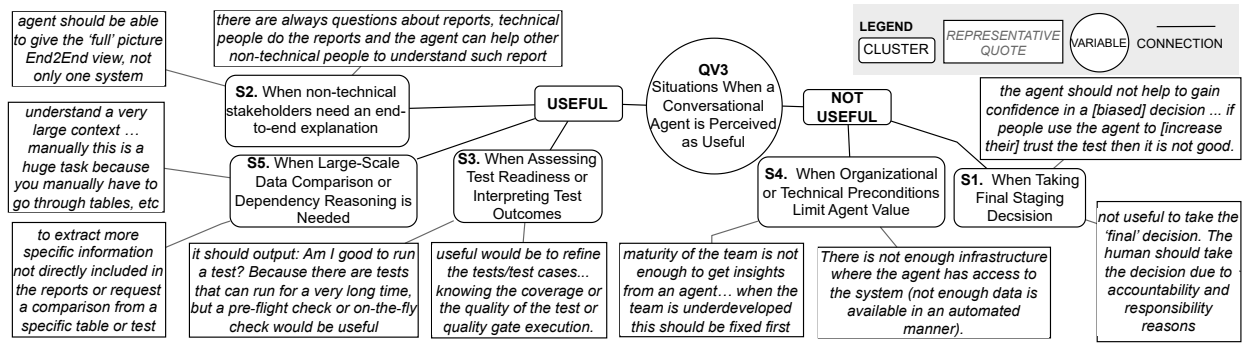


Fig. 4. QV3 Situations When a Conversational Agent is Perceived as Useful: Clusters and textual quotes.

S2. When non-technical stakeholders need an end-to-end explanation. This cluster captures situations in which the conversational agent is useful for enabling non-technical stakeholders to understand technical reports and in decision-making. It reflects challenges where “technical people do the [decision] reports and the agent can help other non-technical people to understand such report.” It should provide an end-to-end, multi-system perspective—“agent should be able to give the ‘full’ picture; End2End view, not only one system”—connecting layers of infrastructure, business logic, and cross-application dependencies.

S3. When Assessing Test Readiness or Interpreting Test Outcomes. This cluster captures situations in which the conversational agent is useful for assessing test readiness prior to execution and for interpreting results afterwards. Subjects consider the conversational agent as useful, especially when long-running or computationally expensive tests are involved, as “tests can run for a very long time, but a pre-flight check or on-the-fly check would be useful.” The subjects bound the perceived usefulness in “test” rather into other data, such as requirements, since “requirements have been decided earlier... these questions are not reevaluated again; agents should be used during testing based on the requirements for the quality gate [i.e., a predefined checkpoint with a set of specified quality criteria that the system must meet] defined previously.”

S5. When Large-Scale Data Comparison or Dependency Reasoning is Needed. This cluster captures situations in which the conversational agent is particularly useful for extracting insights from large, complex, or distributed data sets, where manual analysis would be slow, error-prone, or expensive. Subjects expect the agent “to extract specific information or request a comparison from a specific table or test.” The agent must be capable of interpreting results in a broad context—“understand a very large context; manually doing this is a huge task because you have to go through tables, etc.” This includes analyzing trends over time and connecting outcomes to historical baselines from previous decisions in the staging process.

2) Perceived as Not Useful

S1. When Taking the Final Decision. This cluster reflects that the conversational agent is not useful to make, confirm,

or influence final decision-making in the software staging process. Final responsibility is reserved for humans due to organisational, ethical, and accountability requirements. As mentioned by the subjects, the agent is “not useful to take the ‘final’ decision. The human should take the decision due to accountability and responsibility reasons.” The agent must not participate in automated decision-making workflows that could bypass human accountability, particularly because “automated decisions (in workflow)” are perceived as inappropriate near production.

S4. When Organisational or Technical Preconditions Limit Value. This cluster captures situations in which the usefulness of the conversational agent is limited by external organizational, infrastructural, or contextual factors. A primary constraint is team maturity, as “when the team is underdeveloped, this should be fixed first.” The agent’s effectiveness is also reduced when “there is not enough infrastructure where the agent has access to the system [i.e., staging environment] (not enough data is available in an automated manner)”. Achieving value requires that “a certain set of dependencies are met to give proper ROI [Return on Investment],” since “a low maturity gives bad results and the agent should help to also assess this maturity... goes against the ‘trust’ of the data.”

B. Conversational Agent Design

Conversational Agent Goal Model. We propose an iStar 2.0 (*i**) goal model [8] to design the conversational agent, aligning perceived usefulness in industry (QV3) with functional (QV1) and non-functional (QV2) requirements for the agent. The *i** modelling language is a goal-oriented approach for representing the intentional and strategic relationships of socio-technical systems, capturing stakeholders’ goals (why), dependencies (who), and alternative means for achieving them (how) [8]. Using *i**, we instrumentalise and integrate the knowledge extracted from our study.

We trace each cluster and its key findings into the goal model in Table I ordered by priority. Figure 5 presents the *i** goal model capturing the intentional and strategic relationships associated with using a conversational agent to assist decision-making in software staging processes. The root goal, “decision-making in software staging processes is assisted”, is

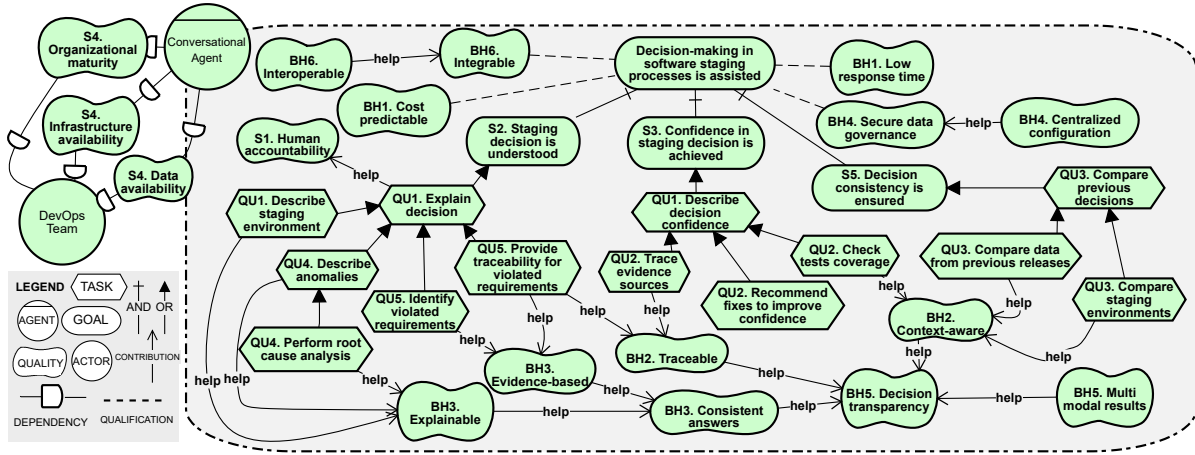


Fig. 5. Goal model for a conversational agent for assisting decision-making in software staging processes

decomposed into the three situations (i.e., goals) that practitioners identified as the primary value points for such an agent (S2, S3, and S5) based on QV3. These situations collectively represent the motivational layer—i.e., the ‘why’—for using a conversational agent to assist with software staging.

Below the goals, the model specifies the concrete tasks required to satisfy them. These tasks correspond to the questions subjects considered essential—those that the agent must or should answer (QU1–QU5)—and represent the how dimension of the model. Finally, the model includes the qualities, which serve as non-functional requirements that the conversational agent must/should/could include (BH1–BH5). Together, these qualities complement both the ‘why’ and the ‘how’, ensuring that the model captures both functional and non-functional requirements.

A Multi-Agent Reference Architecture: The case of the AI-SQUARE innovation project Following the Proposed Goal Model. Based on the requirements captured in the goal model, we designed a reference architecture for a multi-agent system capable of assisting decision-making in software staging processes. This architecture represents the industry implementation of our requirements elicitation. Our emphasis is on tasks (functional requirements) from the goal model. Non-functional requirements may vary depending on the underlying technologies chosen for each agent (e.g., the agentic framework of choice to guarantee low response time; the available infrastructure for running the agent). Figure 6 shows the resulting agent design, which follows the model elements by the Google’s Agentic Development Kit (ADK) and its design patterns for composing LLM-based agentic systems [31].

The architecture follows a coordinator pattern in which a central agent handles high-level conversational interaction and delegates specialised tasks to sub-agents. The main AI-SQUARE conversational agent orchestrates message routing among three agents: i) the decision explanation agent; ii) the decision confidence evaluator agent; and iii) the decision

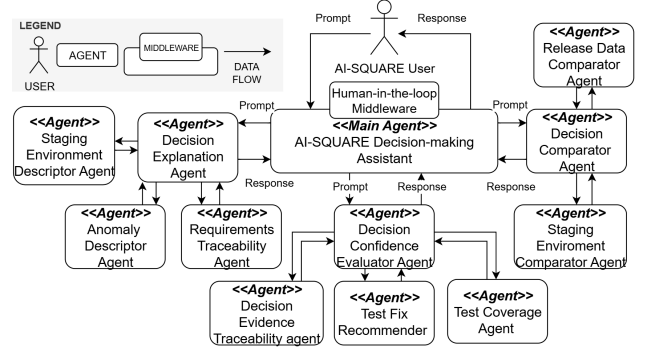


Fig. 6. Multi-agent LLM-based reference architecture for the AI-SQUARE Platform

comparator agent. Each agent aims to address one of the goals identified in our requirements elicitation from industry experts. The **decision explanation agent** is responsible for describing anomalies and their root causes, staging environments and their data, and providing traceability to requirements, enabling decision explanations during the staging process. The **decision confidence evaluator agent** is responsible for answering questions related to confidence in test results, staging-decision justification based on risks and evidence, and recommending fixes to improve the confidence to make a decision. Finally, the **decision comparator agent** is responsible for guaranteeing decision consistency, allowing for the comparison of staging environments and previous releases’ data.

Together, these sub-agents implement the functional requirements identified in the goal model for the AI-SQUARE platform as an instance of our goal model. This architecture is extensible and allows adding more sub-agents as more tasks, goals, and qualities are extended from the identified goals. As an implementation effort, it requires defining the prompts and skills of each agent to achieve their respective goals.

In future work, we will validate the proposed architecture through quasi-experimental studies with industry experts using

an implementation of the conversational agent derived from the proposed architecture. These studies could assess efficiency, effectiveness, and user satisfaction, providing evidence of the agent’s potential adoption and practical utility. Analysing these results would also enable an assessment of how well the goals, qualities, and tasks in the proposed goal model are achieved in practice, as outlined in the proposed multi-agent architecture. Based on the architecture evaluations, we can identify goal model elements that need to be refined, extended, reassigned to different responsibilities, or complemented with new requirements.

VI. THREATS TO VALIDITY

We identified threats to validity during the focus group study. *Regarding internal validity*, a potential threat was the dominance of certain participants, which could lead to the underrepresentation of quieter voices. To mitigate this, the moderator encouraged participants to share and counter each other’s ideas. Additionally, sessions held at different times may have introduced fatigue and reduced engagement, despite the use of breaks and collaborative activities. A further threat is related to the participants’ prior knowledge and experience. Participants with diverse backgrounds may have interpreted the concept of “software staging” differently, potentially influencing their feedback. We mitigated this by introducing and motivating the session, and by using the AI-SQUARE project as a common framework for understanding the concepts. Moreover, the diversity of backgrounds was also valuable because it helped reveal different perspectives to elicit the requirements.

Regarding conclusion validity, the study is subject to a generalisability threat due to the small sample size (8 subjects), concerning whether the conclusions drawn from the collected data are reasonable, well-supported, and consistent with the participants’ feedback. While this cannot be fully mitigated, participants were industry-experienced and representative of software quality assurance and staging in the AI-SQUARE context. Still, we gathered 135 individual requirements for our clustering. Thus, our conclusions and clusters should be interpreted as preliminary and formative, future work should include replications with larger and more diverse populations until saturation is achieved.

Concerning construct validity, threats arise from how well elicited clusters and MoSCoW priorities reflect the intended constructs. Concepts such as usefulness and trustworthiness may be interpreted differently across subjects. We mitigated this by providing context and examples, and by systematically coding and cross-checking interpretations. However, reliance on text-quotes, qualitative coding, and MoSCoW ratings introduces mono-method bias, capturing perceived rather than actual behaviour. Future work should triangulate these constructs using complementary methods (e.g., quasi-experiments). Moreover, we used the MoSCoW method to allow participants to prioritise their contributions, which were later transformed into numeric values during the data analysis. Although this approach was suitable for supporting the

prioritisation of the collected contributions, we acknowledge that using a different requirements prioritisation method could have led to different priority assignments and, consequently, to a different interpretation of the relative importance of the resulting requirements. To mitigate this threat, clustering contributions was based solely on participants’ text content, not on assigned priorities. We used the priority values to support the presentation and interpretation of the results described in the results section. In addition, when constructing the proposed goal model, we included three of the four MoSCoW categories (Must, Should, and Could) and excluded only the “Will not” category.

VII. CONCLUSIONS AND FUTURE STEPS

This paper presents a set of functional and non-functional requirements expected by industry for a conversational agent for assisting decision-making in software staging processes. Through focus groups with experienced SRE, QA, and performance engineering practitioners, we elicited questions, expected behaviours, and situations where a conversational agent is useful or not. The 135 individual requirements were analysed using topic-based modelling, yielding 17 clusters.

Our findings show that DevOps teams in the industry do not seek fully automated decision-making in software staging processes. Instead, they seek transparent, evidence-based support that improves decision quality while maintaining human accountability. Specifically, DevOps teams expect conversational agents to assist them in three main goals related to decision-making in software staging processes: i) understanding a decision; ii) achieving confidence in a decision; and iii) ensuring consistency among decisions based on previous releases. Moreover, explainability, traceability, evidence-based responsiveness, cost predictability, security, data governance, and interoperability are the most important non-functional requirements for such a conversational agent.

To design a conversational agent that meets these industry-driven requirements, we propose a goal model and a reference multi-agent architecture within the AI-SQUARE innovation project. The proposed goal model and multi-agent architecture offer a technology-agnostic foundation for developing LLM-based conversational agents aligned with real industry needs and constraints to assist decision-making in software staging processes. As future work, we aim to develop a proof-of-concept integrated into the AI-SQUARE platform. This prototype will be evaluated through empirical studies to assess its effects on the accuracy, efficiency, and user satisfaction of staging decisions, motivating a use case study. Open questions remain for the requirements engineering community, particularly regarding the extent to which conversational agents for assisting decision-making in software staging can benefit from requirements engineering tools to meet the requirements from industry—such as requirements traceability, teams and organisational maturity, and evidence-based decision-making. These future steps establish a foundation for next-generation, explainable, and human-centred assistance in software staging.

TABLE I: Traceability matrix from Focus Group Results and Proposed Goal model.

Cluster	Key Findings and Priority	Goal Model (i*)
S2. When non-technical stakeholders need an end-to-end explanation	The conversational agent is useful when non-technical stakeholders need end-to-end explanations, stakeholder-specific summaries, or management-ready interpretations of the decision in the staging process.	Goal: Staging decision is understood
S3. When assessing test readiness or interpreting test outcomes	The conversational agent is useful when assessing test readiness or interpreting test outcomes, refining test suites, evaluating coverage, and reducing manual effort.	Goal: Confidence in staging decision is achieved
S5. When large-scale data comparison or dependency reasoning is needed	The conversational agent is useful when large-scale data comparison or dependency reasoning is needed, automating analysis and cross-system correlations among releases.	Goal: Decision consistency is ensured
S1. When taking final staging decision	The conversational agent is not useful for taking automatically final staging decisions, which must remain under human accountability.	Quality: Human accountability
S4. When organisational or technical preconditions limit agent value	The conversational agent is not useful when data access, infrastructure maturity, or team maturity are insufficient, limiting the value it can provide.	Dependum: Infrastructure availability Dependum: Data availability Dependum: Organizational maturity
QU3. How does this release perform compared to previous ones under comparable conditions?	The conversational agent must provide contextual comparison of new versus past releases using valid baselines and must distinguish environment-driven variations from true performance changes.	Task: Compare previous decisions Task: Compare data from previous releases Task: Compare staging environments
QU1. Are we good to go and how confident is this staging decision?	The conversational agent should provide clarity on why the system is (not) ready, which data influenced the decision, which metrics are missing or not applicable, and recommended next steps, mitigation actions, and “what-if” impacts to support human accountability.	Task: Explain decision; Task: Describe decision confidence Task: Describe staging environment (e.g., metrics, logs, etc)
QU2. Can the test results be trusted for this staging decision?	The conversational agent should provide traceability to evidence sources, clarify whether the correct functionality was tested, evaluate data reliability, identify coverage gaps, and recommend test re-runs when results may be biased.	Task: Trace evidence sources Task: Check tests’ coverage Task: Recommend fixes to improve confidence
QU4. Where are the issues, what is causing them, and what qualifies as an anomaly?	The conversational agent should identify anomalies, define them in context, map them to affected components or teams, and provide high-level root-cause reasoning without performing code-level analysis.	Task: Describe anomalies Task: Perform root cause analysis
QU5. Which service level objectives are violated, by whom, and what evidence supports this?	The conversational agent should explain why requirements or SLAs are not met, determine whether performance/load obligations are satisfied, and provide evidence traceability linking violations to underlying measurements and artefacts.	Task: Identify violated requirements Task: Provide traceability for violated requirements
QU6. Which technical choices and deployments are appropriate for staging?	The conversational agent will not provide coding advice, technical implementation suggestions, or other low-level engineering recommendations that fall outside the decision-making assistance for software staging processes.	–
BH3. Explainable, evidence-based, and consistent decision assistance	The conversational agent must deliver role-appropriate explanations grounded in evidence, avoid hallucinations, acknowledge uncertainty, and provide consistent answers to semantically similar questions.	Quality: Explainable Quality: Evidence-based Quality: Consistent answers
BH1. Responsiveness, efficiency and cost predictability	The conversational agent should deliver timely responses with progress feedback, provide measurable efficiency gains such as draft reports, and maintain stable and predictable compute and cost usage.	Quality: Low response time Quality: Cost predictable
BH2. Traceable quality evaluation and context-aware data handling	The conversational agent should perform multi-level coverage evaluation (requirements, tests, data, code, components) and interpret results in context, including environmental, business, and infrastructure changes.	Quality: Traceable Quality: Context-aware
BH4. Secure data governance and controlled configuration	The conversational agent should enforce least-privilege data access, maintain strong data isolation, and operate under centralised, consistent configuration with reversible actions.	Quality: Secure data governance Quality: Centralized configuration
BH6. Enterprise interoperability and collaboration automation	The conversational agent should integrate with enterprise systems such as ticketing and notification platforms and interoperate with other agents in the ecosystem.	Quality: Interoperable Quality: Integrable
BH5. Interaction transparency and multi-modal result representation	The conversational agent could produce multi-modal outputs including graphs, tables, and mixed-media explanations, with interactive handling of outliers, waivers, and cleanup options.	Quality: Multi-modal results Quality: Decision transparency

REFERENCES

- [1] S. D. Network, “The ai-square platform,” Ai-square.ch, [Accessed 10-03-2026].
- [2] L. Leite, C. Rocha, F. Kon, D. Milojicic, and P. Meirelles, “A survey of devops concepts and challenges,” *ACM Computing Surveys*, vol. 52, no. 6, p. 1–35, Nov. 2019. [Online]. Available: <http://dx.doi.org/10.1145/3359981>
- [3] M. Senapathi, J. Buchan, and H. Osman, “Devops capabilities, practices, and challenges: Insights from a case study,” in *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018*, ser. EASE’18. ACM, Jun. 2018, p. 57–67. [Online]. Available: <http://dx.doi.org/10.1145/3210459.3210465>
- [4] G. Melo, P. Alencar, and D. Cowan, “Enhancing software development with context-aware conversational agents: A user study on developer interactions with chatbots,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.08648>
- [5] J. H. Klemmer, S. A. Horstmann, N. Patnaik, C. Ludden, C. Burton, C. Powers, F. Massacci, A. Rahman, D. Votipka, H. R. Lipford, A. Rashid, A. Naiakshina, and S. Fahl, “Using ai assistants in software development: A qualitative study on security practices and concerns,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’24. ACM, Dec. 2024, p. 2726–2740. [Online]. Available: <http://dx.doi.org/10.1145/3658644.3690283>
- [6] D. Mosquera, M. Ruiz, O. Mann, and M. Hanin, “Ai-square: Knowledge graphs for requirements-driven software staging management,” in *Joint Proceedings of REFSQ-2025 Workshops, Doctoral Symposium, Posters & Tools Track, and Education and Training Track. Co-located with REFSQ 2025*. Aachen, Germany: CEUR, 2025. [Online]. Available: <https://ceur-ws.org/Vol-3959/PT-paper3.pdf>
- [7] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, p. 1–79, Nov. 2024. [Online]. Available: <http://dx.doi.org/10.1145/3695988>
- [8] F. Dalpiaz, X. Franch, and J. Horkoff, *Definition and Use of iStar 2.0*. Switzerland: Springer Nature, 2024, pp. 65–81.
- [9] H. E. Korkmaz and M. N. Aydin, “An empirical study on performance comparisons of different types of devops team formations,” *Frontiers in Computer Science*, vol. 7, Sep. 2025. [Online]. Available: <http://dx.doi.org/10.3389/fcomp.2025.1554299>
- [10] T. Olsson, K. Wnuk, and T. Gorschek, “An empirical study on decision making for quality requirements,” *Journal of Systems and Software*, vol. 149, p. 217–233, Mar. 2019. [Online]. Available: <http://dx.doi.org/10.1016/j.jss.2018.12.002>
- [11] C.-P. Bezemer, S. Eismann, V. Ferme, J. Grohmann, R. Heinrich, P. Jamshidi, W. Shang, A. van Hoorn, M. Villavicencio, J. Walter, and F. Willnecker, “How is performance addressed in devops?” in *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE ’19. ACM, Apr. 2019, p. 45–50. [Online]. Available: <http://dx.doi.org/10.1145/3297663.3309672>
- [12] R. Amaro, R. Pereira, and M. M. da Silva, “Capabilities and practices in devops: A multivocal literature review,” *IEEE Transactions on Software Engineering*, vol. 49, no. 2, p. 883–901, Feb. 2023. [Online]. Available: <http://dx.doi.org/10.1109/TSE.2022.3166626>
- [13] S. I. Abbas and A. Garg, “Aiops in devops: Leveraging artificial intelligence for operations and monitoring,” in *2024 3rd International Conference on Sentiment Analysis and Deep Learning (ICSADL)*. IEEE, Mar. 2024, p. 64–70. [Online]. Available: <http://dx.doi.org/10.1109/ICSADL61749.2024.00016>
- [14] J. Diaz-de Arcaya, A. I. Torre-Bastida, G. Zárate, R. Miñón, and A. Almeida, “A joint study of the challenges, opportunities, and roadmap of mlps and aiops: A systematic survey,” *ACM Computing Surveys*, vol. 56, no. 4, p. 1–30, Oct. 2023. [Online]. Available: <http://dx.doi.org/10.1145/3625289>
- [15] I. Kumara, G. Quattrocchi, D. Tamburri, and W.-J. Van Den Heuvel, “Quality assurance of heterogeneous applications: The SODALITE approach,” in *Advances in Service-Oriented and Cloud Computing*, 2021, p. 173–178.
- [16] J. Qiu, Q. Du, K. Yin, S.-L. Zhang, and C. Qian, “A causality mining and knowledge graph based method of root cause diagnosis for performance anomaly in cloud applications,” *Applied Sciences*, vol. 10, no. 6, 2020.
- [17] H. Wang, Z. Wu, H. Jiang, Y. Huang, J. Wang, S. Kopru, and T. Xie, “Groot: An event-graph-based approach for root cause analysis in industrial settings,” in *36th International Conference on Automated Software Engineering*, 2021, p. 419–429.
- [18] T. Wang, G. Qi, and T. Wu, “Kgroot: A knowledge graph-enhanced method for root cause analysis,” *Expert Systems with Applications*, vol. 255, p. 124679, 2024.
- [19] Dynatrace, “Grail,” <https://www.dynatrace.com/platform/grail/>, 2025, [Accessed 07-02-2025].
- [20] J. He, C. Treude, and D. Lo, “Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, p. 1–30, May 2025. [Online]. Available: <http://dx.doi.org/10.1145/3712003>
- [21] Y. Cai, R. Li, P. Liang, M. Shahin, and Z. Li, “Designing llm-based multi-agent systems for software engineering tasks: Quality attributes, design patterns and rationale,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.08475>
- [22] A. G. Khoei, Y. Yu, R. Feldt, A. Freimanis, P. A. Rhodin, and D. Parthasarathy, *GoNoGo: An Efficient LLM-Based Multi-agent System for Streamlining Automotive Software Release Decision-Making*. Springer Nature Switzerland, 2025, p. 30–45. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-80889-0_3
- [23] A. G. Khoei, S. Wang, Y. Yu, R. Feldt, and D. Parthasarathy, “Gatelens: A reasoning-enhanced llm agent for automotive software release analytics,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.21735>
- [24] R. van Solingen, V. Basili, G. Caldiera, and H. D. Rombach, “Goal question metric (gqm) approach,” Jan. 2002. [Online]. Available: <http://dx.doi.org/10.1002/0471028959.sof142>
- [25] J. Kontio, J. Bragge, and L. Lehtola, *The Focus Group Method as an Empirical Tool in Software Engineering*. Springer London, 2008, p. 93–116. [Online]. Available: http://dx.doi.org/10.1007/978-1-84800-044-5_4
- [26] D. Morgan, *Focus Groups as Qualitative Research*. SAGE Publications, Inc., 1997. [Online]. Available: <http://dx.doi.org/10.4135/9781412984287>
- [27] P. Voola and A. V. Babu, “Comparison of requirements prioritization techniques employing different scales of measurement,” *ACM SIGSOFT Software Engineering Notes*, vol. 38, pp. 1–10, 2013.
- [28] S. Hutton, *Early Prioritisation of Goals*. Springer Berlin Heidelberg, 2007, p. 235–244. [Online]. Available: http://dx.doi.org/10.1007/978-3-540-76292-8_29
- [29] M. Grootendorst, “Bertopic: Neural topic modeling with a class-based tf-idf procedure,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.05794>
- [30] D. Mosquera, M. Ruiz, L. Willie, T. Riedel, O. Mann, and M. Hanin, “Industry-derived requirements for an llm-based conversational agent to assist software staging: Protocol, raw data, scripts, and demographics,” Mar. 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.19088305>
- [31] Google, “Developer’s guide to multi-agent patterns in ADK,” <https://developers.googleblog.com/developers-guide-to-multi-agent-patterns-in-adk/>, [Accessed 10-03-2026].